

```

`timescale 1ns/1ps
module alu_bit(
input [31:0]A,
input [31:0]B,
input [31:0]Alu_Sel,
output reg [31:0]Alu_Out,
output Zero
);

assign Zero=(Alu_Out==32'b0);

always @(*) begin
case(Alu_Sel)

4'b0000:Alu_Out=A+B;
4'b0001:Alu_Out=A-B;
4'b0010:Alu_Out=A&B;
4'b0011:Alu_Out=A|B;
4'b0100:Alu_Out=A^B;
4'b0101:Alu_Out=~A;
4'b0110:Alu_Out=A<<1;
4'b0111:Alu_Out=A>>1;
4'b1000:Alu_Out=(A<B)? 32'd1:32'd0;

default :Alu_Out=32'b0;
endcase
end
endmodule

```

```

-----
`timescale 1ns/1ps
module alu_bit(
input [31:0]A,
input [31:0]B,
input [31:0]Alu_Sel,
output reg [31:0]Alu_Out,
output Zero
);

assign Zero=(Alu_Out==32'b0);

always @(*) begin
case(Alu_Sel)

4'b0000:Alu_Out=A+B;
4'b0001:Alu_Out=A-B;
4'b0010:Alu_Out=A&B;
4'b0011:Alu_Out=A|B;
4'b0100:Alu_Out=A^B;
4'b0101:Alu_Out=~A;
4'b0110:Alu_Out=A<<1;
4'b0111:Alu_Out=A>>1;
4'b1000:Alu_Out=(A<B)? 32'd1:32'd0;

default :Alu_Out=32'b0;
endcase
end
endmodule
-----

```

```

-----
`timescale 1ns/1ps
module bit_shift_multi (input [3:0] a, input [3:0] b,output [7:0]
product);

    reg [7:0] prod_reg;
    reg [3:0] count;
    integer i;
    always @* begin
    prod_reg=8'b0;
    for (i=0;i<4;i=i+1) begin
    if (b[i] == 1'b1) begin
    prod_reg= prod_reg + (a<<i);
    end
    end
    end
    assign product=prod_reg;
endmodule
-----

`timescale 1ns/1ps
module test_bit_shift_multi;
reg [3:0] a_tb;
reg [3:0] b_tb;
wire [7:0] product_tb;
bit_shift_multi dut (.a(a_tb),.b(b_tb),.product(product_tb));
initial begin
$display("Time\ta\tb|product(Binary)|Product(Decimal)");
$display("-----");

a_tb=4'b0000; b_tb=4'b0000; #10;

$display("%0dns:\t%b\t%b\t|%0d",$time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b0001; b_tb=4'b0001; #10;

$display("%0dns:\t%b\t%b\t|%0d",$time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b0010; b_tb=4'b0011; #10;

$display("%0dns:\t%b\t%b\t|%0d",$time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b1010; b_tb=4'b0011; #10;

$display("%0dns:\t%b\t%b\t|%0d",$time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b1101; b_tb=4'b0011; #10;

$display("%0dns:\t%b\t%b\t|%0d",$time,a_tb,b_tb,product_tb,product_tb);

#10;
$stop;
end endmodule
-----
-----
-----
-----

```

```

module four_bit_adder(a,b,c0,s,c4);
input [3:0]a,b;
input c0;
output [3:0]s;
output c4;
wire c1,c2,c3;
full_adder fa0(a[0],b[0],c0,s[0],c1);
full_adder fa1(a[1],b[1],c1,s[1],c2);
full_adder fa2(a[2],b[2],c2,s[2],c3);
full_adder fa3(a[3],b[3],c3,s[3],c4);
endmodule

```

```

module full_adder(a,b,ci,s,co);
input a,b,ci;
output s,co;
assign s=a^b^ci;
assign co=(a&b)|(b&ci)|(a&ci);
endmodule

```

```

-----
-----

```

```

module test_4_bit;
reg [3:0]a;
reg [3:0]b;
reg c0;
wire [3:0]s;
wire c4;
four_bit_adder dut(a,b,c0,s,c4);
initial begin
a=4'b0011; b=4'b0011; c0=1'b0; #10;
a=4'b0011; b=4'b0111; c0=1'b1; #10;
a=4'b1101; b=4'b0011; c0=1'b1; #10;
end initial
#50 $finish;
endmodule

```

```

-----
-----
-----

```

```

module d_flipflop(D,clk,sync_reset,Q);
input D;
input clk;
input sync_reset;
output reg Q;
always @(posedge clk)
begin
if (sync_reset==1'b1)
Q<=1'b0;
else
Q<=D;
end
endmodule

```

```

-----
-----

```

```

module tb_flipflop();
reg D;
reg clk;
reg reset;
wire Q;

```

```

d_flipflop dut(D,clk,reset,Q);
initial begin
clk=0;
forever#10clk=~clk;
end
initial begin
reset=1;
D<=0;
#100;
reset=0;
D<=1;
#100;
D<=0;
#100;
D<=1;
end
endmodule

```

```

-----
module sr_flipflop(output reg q,qbar,
input s,r,clk
);
always @ (posedge clk)begin
if(s==1 && r==0)begin
q<=1;
qbar<=0;
end
else if(s==0 && r==1)begin
q<=0;
qbar<=1;
end
else if(s==0 && r==0)begin
q<=q;
qbar<=qbar;
end
else if(s==1 && r==1)begin
q<=1'bx;
qbar<=1'bx;
end
end
endmodule

```

```

-----
module tb_sr_flipflop;
reg s,r,clk;
wire q,qbar;
sr_flipflop dut(.q(q),.qbar(qbar),.s(s),.r(r),.clk(clk));
initial begin
clk=0; s=0; r=0;
#10 s=1; r=0;
#10 s=0; r=0;
#10 s=1; r=1;
#10 s=0; r=0;
#50 $finish;
end
always #5 clk=~clk;
endmodule
-----

```

```

module jk_flipflop(input reset,input clk,input j,input k, output reg
q,output qnot);
assign qnot=~q;
always @(posedge clk)
if (reset) q<=1'b0;
else
case({j,k})
2'b00:q<=q;
2'b01:q<=1'b0;
2'b10:q<=1'b1;
2'b11:q<=~q;
endcase
endmodule

```

```

-----
--
module jktb;
reg clk=0;
reg j=0;
reg k=0;
reg reset=0;
wire q,qnot;
jk_flipflop dut(reset,clk,j,k,q,qnot);
initial begin
#10;j=1'b0;k=1'b0;
#10;j=1'b1;k=1'b0;
#10;j=1'b0;k=1'b1;
#10;j=1'b1;k=1'b1;
#10;j=1'b1;k=1'b1;
#25; reset=1'b1;
end
always #1 clk=~clk;
endmodule

```

```

-----
----
module mod_n_count(
input clk,
input rst_n,
input [3:0]N,
output reg [3:0]count
);
always @ (posedge clk or negedge rst_n) begin
if (~rst_n)
count<=4'b0000;
else if (count==N-1)
count<=4'b0000;
else
count<=count+1;
end
endmodule

```

```

-----
module tb_modn_count;
reg clk;
reg rst_n;
reg [3:0]N;
wire [3:0]count;
mod_n_count dut(
.clk(clk),

```

```

.rst_n(rst_n),
.N(N),
.count(count)
);
always begin
#5; clk=~clk;
end
initial begin
clk=0;
rst_n=0;
N=4;
#10;
rst_n=1;
#40;
N=8;
#80;
N=10;
#100;
$finish;
end
initial begin
$monitor ("Time=%0t, clk=%b, rst_n=%b,N%d",$time,clk,rst_n,N,count);
end
endmodule

```

```

-----
-----

```